

CogniTek — Readiness Assessment (1-4-2026)

A comprehensive audit of what stands between our current app and a fully-fledged Google Play Store release.

Summary

Category	Status
Capacitor + Android Shell	✓ Basic setup done
Authentication (Supabase)	✓ Working
Core Features (Record, Tasks, Flashcards, Sylens)	✓ Working
Play Store Publishing Requirements	⦿ Not started
Backend Production Readiness	⦿ Partial
Offline & Resilience	⦿ PWA caching only
Native Android Integration	⦿ Minimal
Testing & Quality Assurance	⦿ None

Critical Blockers (Must Fix Before Publishing)

1. Backend is on ngrok / localhost — No Stable Production URL

⚠ CAUTION: Our app currently points to `http://127.0.0.1:8000` as the default API, with an ngrok tunnel for phone testing. Play Store apps cannot rely on ngrok tunnels or localhost.

What we need:

- A **permanently hosted backend** with a stable URL (e.g., `https://api.cognitek.app`)
- Render.com's **free tier spins down after inactivity** (30-60s cold starts = terrible UX)
- **Options:** Render paid tier (~\$7/mo), Railway.app, Fly.io, or a cheap VPS (DigitalOcean \$6/mo)
- The **Whisper model requirement** makes this harder — needs ~2GB+ RAM

2. Whisper Model Cannot Run on Free Tier Hosting

⚠ WARNING: Our backend loads `whisper.load_model("small")` which requires ~1GB+ RAM and torch. Free-tier services will OOM crash or be painfully slow.

Options:

- Use **Groq's Whisper API** (Already have Groq set up!) — free tier gives decent quotas
- Use **Google Cloud Speech-to-Text** or **AssemblyAI**
- Use **OpenAI Whisper API** (\$0.006/min — very cheap)
- Self-host on a GPU server (expensive for a student project)

3. No App Signing Configuration

i IMPORTANT: Google Play requires a signed APK/AAB (Android App Bundle). Our current setup has no signing config.

What's needed in build.gradle:

- Add a **signingConfigs** block with keystore path, passwords, and key alias
- Generate a keystore with keytool — **never lose it** (you can't update the app without it)
- Enroll in **Google Play App Signing** (recommended)

4. No Privacy Policy

⚠ CAUTION: Google Play rejects apps without a privacy policy. Our app collects: audio recordings, user email, academic data, and uses AI services. This **REQUIRES** a privacy policy.

What you need:

- A publicly hosted privacy policy page (e.g., on GitHub Pages or Vercel site)
- Must disclose: data collected, how it's processed, third-party services (Supabase, Google Gemini, Groq), data retention, user rights
- Link it in the Play Store listing AND inside the app (Profile page)

5. No Proper App Icons & Store Listing Assets

Current state: Our pwa-192x192.png and pwa-512x512.png are 685KB each (likely the same file — probably a placeholder).

Play Store requires:

Asset	Specification
App Icon	512×512 PNG (32-bit, no alpha for Play Store)
Feature Graphic	1024×500 PNG
Screenshots	Min 2 phone screenshots (16:9 or 9:16)
Short Description	Max 80 characters
Full Description	Max 4000 characters
App Category	Education
Content Rating	Complete IARC questionnaire

Android adaptive icons need to be generated from our logo with multiple density variants (mdpi → xxxhdpi).

6. Version Code/Name Not Managed

In build.gradle: versionCode 1, versionName "1.0"

- Each Play Store update must have a **higher versionCode**
- You need a versioning strategy (e.g., semver: 1.0.0, 1.1.0, 1.2.0)

Important Improvements (Should Fix)

7. No Offline Capability for the Native App

Our PWA has Workbox caching, but as a Capacitor native app, the service worker approach is less reliable.

What We need:

- Local storage/SQLite for offline task/flashcard viewing
- Queue system for audio uploads when offline (record → store locally → upload when back online)
- Graceful error handling when backend is down (currently shows "Connection Failed" with no retry)

8. Missing Native Permissions Handling

AndroidManifest.xml declares RECORD_AUDIO and CAMERA, but:

- **No runtime permission request flow** — Android 6+ requires asking at runtime
- AudioRecorder.jsx uses navigator.mediaDevices.getUserMedia() — works in WebView but error handling is minimal
- We should use **@capacitor/permissions** or **@capacitor/microphone** plugin for proper native permission flows
- **CAMERA** permission is declared but no camera feature exists — Google may flag this as unnecessary

9. No Push Notifications

- The google-services.json hook exists in build.gradle but the file itself is missing
- No Firebase/FCM setup
- For an academic app, notifications for **upcoming deadlines** and **study reminders** would be expected

10. Backend Security Issues

 **WARNING:** Backend has allow_origins=["*"] — this allows any website to call your API.

Additional concerns:

- No rate limiting — anyone can spam your Gemini/Groq endpoints and burn your API quota
- No authentication on API endpoints — anyone with your URL can read/write ANY user's data by guessing UUIDs
- API keys read from plaintext files (api_key.txt, groq_key.txt) — should be environment variables only
- No input validation on audio file uploads (size limits, file type verification)
- The temp_filename in process_audio could be a path traversal vulnerability

11. Deep Linking / Navigation Issues

- Using BrowserRouter which requires server-side routing config
- Capacitor apps sometimes have issues with BrowserRouter — consider **HashRouter**
- No Android deep link / App Link configuration

12. No Error Reporting / Crash Analytics

- ErrorBoundary shows a dev-friendly crash screen — not user-friendly for production
- No crash reporting service (Sentry, Firebase Crashlytics, etc.)
- No analytics to understand user behavior

Nice-to-Have (Polish for Better Store Rating)

13. No Loading States / Skeleton Screens

- When the app loads data, users see nothing or brief flashes — skeleton loaders would feel more polished

14. No Data Export / Account Deletion

- Google Play requires apps to offer **account deletion** if they support account creation
- Users should be able to export their data (GDPR-like compliance)

15. Haptic Feedback

- Recording button should vibrate on tap (native feel)
- Use `@capacitor/haptics` plugin

16. App Size Optimization

- Current Capacitor build likely includes all of Tailwind CSS — tree-shake unused styles
- Lazy-load routes to reduce initial bundle

17. Accessibility

- No ARIA labels on interactive elements
- No screen reader support
- Color contrast may not meet WCAG guidelines in some dark-mode elements

18. No Automated Testing

- Zero unit tests, integration tests, or E2E tests
- Play Store doesn't require them, but any update could break existing functionality

Recommended Action Plan (Priority Order)

Follow this sequence to get to Play Store as efficiently as possible:

1. **Deploy Backend to Paid Hosting** — *Render paid tier or Railway — ~\$7/mo*
2. **Replace Whisper with Cloud STT API** — *Use Groq Whisper API (key already set up)*
3. **Add API Authentication & Rate Limiting** — *Secure endpoints, restrict CORS*
4. **Generate Signing Key & Build Release AAB** — *keytool + Gradle signing config*
5. **Create Privacy Policy Page** — *Host on GitHub Pages or Vercel*
6. **Generate Proper App Icons & Store Assets** — *Adaptive icons, screenshots, feature graphic*
7. **Create Google Play Developer Account** — *\$25 one-time fee*
8. **Complete Store Listing & Content Rating** — *Description, category, IARC questionnaire*
9. **Submit for Review** — *Upload AAB, publish*

Estimated Effort

Task	Time Estimate
Backend hosting + Whisper migration	1-2 days
API security (auth, rate limiting)	1 day
App signing + release build	2-3 hours
Privacy policy	1-2 hours
Store assets (icons, screenshots, description)	3-4 hours
Play Store account + listing setup	1-2 hours
Account deletion feature	2-3 hours
Total minimum	~3-4 days

Costs to Consider

Item	Cost
Google Play Developer Account	\$25 (one-time)
Backend Hosting (Render/Railway paid)	~\$7-15/month
Cloud STT API (Groq Whisper / OpenAI)	Free tier likely sufficient
Gemini API	Free tier likely sufficient
Custom Domain (optional)	~\$10-15/year
Total upfront	~\$30-40
Total monthly	~\$7-15

💡 **TIP:** The fastest path to Play Store: migrate audio transcription to Groq Whisper API (you already have the key), deploy backend to Render paid tier, add a privacy policy, generate signing keys, and submit. The core app features are already solid — it's mostly infrastructure and compliance work.